Hochschule Karlsruhe
University of
Applied Sciences

Fakultät für
Informatik und
Wirtschaftsinformatik

Laborbericht

Optimierung eines Raytracers

im Studiengang
Informatik Master

vorgelegt von

Florian Kaiser
Matrikelnummer: 79728
kafl1018@h-ka.de

an der Hochschule Karlsruhe
am 03.11.2026

Prüfer: Prof. Dr. Christian Pape

1 Einleitung

Ein Raytracer berechnet ein Bild, indem er durch jedes Pixel einen Sehstrahl von der Kamera in die Szene schickt und das erste Objekt sucht, das dieser Strahl trifft. Im Trefferpunkt wird die Helligkeit bestimmt. Ob die Lichtquelle von dort aus sichtbar ist, prüft ein weiterer Strahl, der Schattenstrahl. Beim Verfahren nach Whitted [1] werden an spiegelnden und durchsichtigen Oberflächen zudem rekursiv ein reflektierter und ein gebrochener Strahl verfolgt. Fast die gesamte Rechenzeit entfällt dabei auf die Frage, welches Dreieck ein Strahl als erstes trifft. An dieser Stelle setzen alle Aufgaben der Rechnerübung an.

Gemessen wird ausschließlich mit der vorgegebenen Testszene `teapot_n_glass.obj`. Sie zeigt eine Teekanne, drei Töpfe aus Glas, Porzellan und Ton sowie eine runde Bodenplatte aus zusammen 18 032 Dreiecken. In der ersten Aufgabe wird jeder Strahl gegen alle Dreiecke geprüft. Die zweite ersetzt diese Suche durch einen k-d-Baum und die dritte verfolgt mehrere Strahlen gemeinsam mit AVX-Befehlen. Jede Aufgabe wird im Folgenden in einem eigenen Kapitel mit Entwurf, Bildergebnissen und Zeitmessungen beschrieben.

1.1 Messmethodik

Damit die Zeiten aller Aufgaben vergleichbar bleiben, legt das Programm Kamera und Lichtquelle allein aus der Szene fest (Abschnitt 2.1). Die Zeiten misst das Programm selbst mit `std::chrono::steady_clock`, getrennt für das Einlesen der Szene und das Rendern. Zusätzlich zählt es die Strahlen jeder Art und die Schnitttests. Das Skript `messung/messen.sh` baut den Raytracer im Release-Modus mit `-O3`, startet einen Aufwärmlauf und misst danach jede Konfiguration dreimal. Jeder Lauf landet als Zeile in einer CSV-Datei, aus der die Tabellen beim Setzen des Berichts berechnet werden. Angegeben ist, wenn nicht anders vermerkt, der Median der drei Läufe. Gerendert wird in einem einzigen Thread. Da `rand()` ohne eigenen Startwert läuft, erzeugt jeder Lauf einer Konfiguration dasselbe Bild und dieselben Zählerstände. Gemessen wurde auf einem MacBook Pro mit Apple M1 und 16 GB Arbeitsspeicher unter macOS 26.5, übersetzt mit Apple Clang 21.0.0.

Für Teile des Programmcodes, die Unit-Tests, das Messskript und Textentwürfe wurde Claude (Anthropic) eingesetzt. Code, Messwerte und Text wurden vom Autor geprüft.

2 Aufgabe 1: Brute-Force-Raytracer

In der ersten Aufgabe entsteht ein Whitted-Raytracer ohne Beschleunigungsstruktur, der jeden Strahl gegen alle Dreiecke der Szene testet. Er dient als Referenz für die folgenden Aufgaben. Im Mittelpunkt stehen die beiden Schnitttests zwischen Strahl und Dreieck nach Badouel und nach Möller und Trumbore, die über die Option `--badouel` gegeneinander ausgetauscht werden.

2.1 Programmentwurf

Grundlage ist die Quelltextbasis aus dem Computergrafiklabor. Das Programm ist in C++20 geschrieben, wird mit CMake gebaut und braucht außer GoogleTest keine Bibliotheken. Die Klassen liegen in fünf aufeinander aufbauenden Verzeichnissen. `math/` enthält die Vektorklasse und `geometry/` Strahl, Kugel und Dreieck mit ihren Schnitttests. Beide sind Templates über Zahlentyp und Dimension, verwendet werden dreidimensionale Vektoren aus `float`. `world/` fasst Objekte, Materialien und Lichtquellen in der Klasse `World` zusammen. In `view/` erzeugen Kamera, Bildebene und Fenster aus der Welt das Bild und `utility/` enthält den Wavefront-Import und die BMP-Ausgabe. Alles, was von der Art der Suche abhängt, liegt in `World`. Die Kamera ruft nur deren zwei Suchen auf und bleibt deshalb unverändert, wenn in den folgenden Aufgaben eine Beschleunigungsstruktur hinzukommt.

Ein Vektor belegt 12 Byte, ein Dreieck aus drei Eckpunkten und drei Eckpunktnormalen 72 Byte. Die Szene ist ein `std::vector`, in dem die Objekte als Werte hintereinander liegen. Ein Objekt enthält wie im Labor sowohl eine Kugel als auch ein Dreieck und ein Feld, das angibt, welches davon gilt. Das kommt ohne virtuelle Methoden aus, kostet aber 112 Byte je Objekt, obwohl der Schnitttest nur die 36 Byte der Eckpunkte braucht. Die rund 2 MB der Testszene werden bei der Brute-Force-Suche für jeden Strahl einmal vollständig durchlaufen.

Die Suche `find_nearest_object` testet jedes Objekt und merkt sich den nächsten Treffer. Erst für diesen wird die Normale aus den Eckpunktnormalen interpoliert. Die Suche `ray_intersects_any` dient dem Schattentest und bricht beim ersten Treffer zwischen Punkt und Licht ab. Der Aufwand beider Suchen wächst damit mit dem Produkt aus der Zahl der Strahlen und der Zahl der Dreiecke. Für jedes Pixel erzeugt das Fenster einen oder mehrere Sehstrahlen an zufälligen Stellen im Pixel und mittelt ihre Farben. Die Kamera sucht für jeden Strahl den nächsten Treffer und verzweigt nach dem Material. Spiegel und Glas erzeugen rekursiv Folgestrahlen bis zur Tiefe 5, ein Strahl ohne Treffer liefert einen hellblauen Himmel. Das fertige Bild wird als 24-Bit-BMP geschrieben.

Der Wavefront-Import liest Eckpunkte, Normalen und Flächen in einem Durchlauf, zerlegt Polygone in Dreiecke und übernimmt die Eckpunktnormalen, sodass die Teekanne glatt erscheint. Ein durchsichtiges Material wird zu Glas mit dem Brechungsindex aus der Materialdatei. In der Testszene ist damit der Glastopf aus Glas und alles andere diffus. Kamera und Licht bestimmt das Programm aus dem Quader um alle Dreiecke. Die Kamera steht bei (0; 8,47; −12,02) auf der Seite mit negativem z, da von der anderen Seite die Teekanne zwei der Töpfe verdeckt. Sie blickt aus 30° Höhe mit 40° Öffnungswinkel auf die Mitte der Szene. Die Punktlichtquelle steht bei (1,36; 6,96; −2,71) über der Szene.

Die 109 Unit-Tests mit GoogleTest liegen neben dem getesteten Code. Einer schickt 2000 Zufallsstrahlen in eine Szene aus 200 zufälligen Dreiecken und verlangt von beiden Schnittverfahren denselben Treffer im selben Abstand. Ein anderer zählt die Schnitttests und prüft so den Abbruch des Schattentests.

2.2 Schnittverfahren

Beide Verfahren bestimmen für einen Strahl und ein Dreieck den Abstand des Treffers und zwei baryzentrische Koordinaten, mit denen die Normale interpoliert wird. Beide schneiden auch Rückseiten, da die Glastöpfe von innen getroffen werden.

Das Verfahren nach Badouel [2] schneidet zuerst den Strahl mit der Ebene des Dreiecks und berechnet daraus den Schnittpunkt. Danach wird geprüft, ob dieser Punkt im Dreieck liegt. Badouel projiziert dazu Punkt und Dreieck auf eine Koordinatenebene und rechnet dort in zwei Dimensionen weiter. Die Umsetzung aus dem Labor prüft stattdessen im Raum mit drei Kreuzprodukten, ob der Punkt für jede Kante auf der Innenseite liegt. Die baryzentrischen Koordinaten ergeben sich dort aus den Flächen der Teildreiecke.

Das Verfahren nach Möller und Trumbore [3] beschreibt den Trefferpunkt direkt über zwei Kanten des Dreiecks und löst das entstehende Gleichungssystem mit der Cramerschen Regel. Ebene und Schnittpunkt werden nicht gebraucht und das Verfahren kann schon nach der ersten und nach der zweiten baryzentrischen Koordinate abbrechen (Listing 1). Da bei der Brute-Force-Suche fast jeder Test sein Dreieck verfehlt, spart das den größten Teil der Arbeit. Bis zur ersten Abbruchprüfung braucht Möller-Trumbore 13 Multiplikationen und eine Division. Die Variante nach Badouel braucht bis dahin 30 Multiplikationen, eine Division und eine Wurzel für die Länge der Normale.

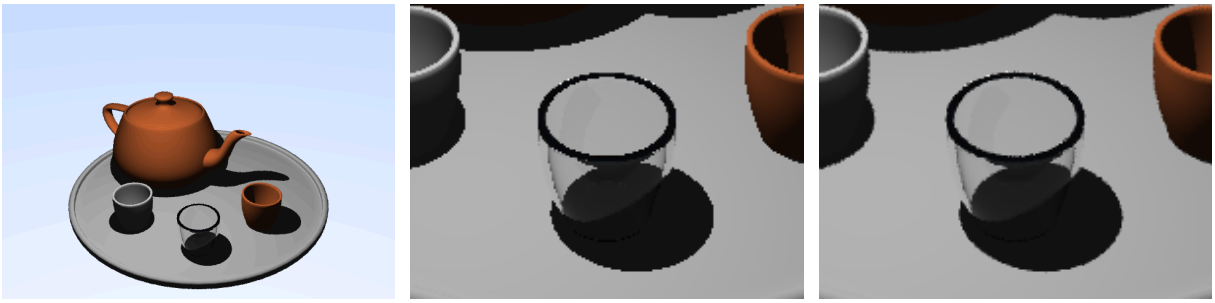
```
Vector<FLOAT, N> edge1 = b - a, edge2 = c - a;
Vector<FLOAT, N> p_vec = ray.direction.cross_product(edge2);
FLOAT det = edge1 * p_vec;
if ( fabs(det) < EPSILON ) return false;               // Strahl parallel zum Dreieck
FLOAT inv_det = 1.0 / det;
Vector<FLOAT, N> t_vec = ray.origin - a;
FLOAT bary_b = (t_vec * p_vec) * inv_det;               // Gewicht von b
if ( bary_b < 0.0 || bary_b > 1.0 ) return false;
Vector<FLOAT, N> q_vec = t_vec.cross_product(edge1);
FLOAT bary_c = (ray.direction * q_vec) * inv_det;       // Gewicht von c
if ( bary_c < 0.0 || bary_b + bary_c > 1.0 ) return false;
FLOAT t = (edge2 * q_vec) * inv_det;
if ( t < 0.0 ) return false;
```

Listing 1: Schnitttest nach Möller und Trumbore (`geometry/geometry.tcc` , gekürzt)

2.3 Beleuchtung und Materialien

Diffuse Flächen werden nach Lambert beleuchtet. Zum Kosinus zwischen Normale und Richtung zum Licht kommt ein ambienter Anteil von 0,1, der überall gilt. In einer früheren Fassung erhielten vom Licht abgewandte Flächen keinen ambienten Anteil. An der Teekanne entstand dadurch ein treppenförmiger Streifen an der Schattengrenze. Der Schattenstrahl startet leicht entlang der Normale versetzt. Seine Richtung ist der unnormierte Vektor zum Licht, sodass jeder Treffer vor dem Licht Schatten wirft. Zeigt die Fläche vom Licht weg, entfällt er ganz. Glas erzeugt einen gebrochenen Strahl nach dem Brechungsgesetz von Snellius und einen reflektierten Strahl. Ihre Farben werden nach der Näherung von Schlick [4] für den Fresnel-Faktor gemischt. Ob ein Strahl in das Glas eintritt oder es verlässt, entscheidet die Orientierung des Dreiecks. Ist keine Brechung möglich, wird der Strahl total reflektiert. Glas wirft wie jedes andere Objekt einen vollen Schatten.

2.4 Bildergebnisse



(a) 640 × 480, 4 Sehstrahlen (b) Ausschnitt, 1 Sehstrahl (c) Ausschnitt, 4 Sehstrahlen

Abbildung 1: Testszene mit Möller-Trumbore und vergrößerter Ausschnitt um den Glastopf

Abbildung 1 zeigt die Testszene mit der festgelegten Kamera. Zu erkennen sind die harten Schatten auf der Platte und die Brechung im Glastopf, durch den Platte und Rand verzerrt erscheinen. Im vergrößerten Ausschnitt entstehen mit einem Sehstrahl je Pixel an den Rändern und Schattengrenzen Treppenstufen. Mit vier Strahlen an zufälligen Stellen im Pixel werden die Übergänge weicher. Die Bilder beider Schnittverfahren sind bei 160 × 120 Pixeln gleich. Bei 320 × 240 Pixeln weichen 2 von 76 800 Pixeln ab und bei 640 × 480 Pixeln 9 von 307 200, jeweils um höchstens eine Stufe in einem Farbkanal. Ursache sind Rundungsfehler bei der Interpolation der Normale.

2.5 Zeitmessungen

Vergleich der Schnittverfahren. Tabelle I stellt beide Verfahren in der Referenzkonfiguration gegenüber, also bei 320 × 240 Pixeln, einem Sehstrahl je Pixel und Rekursionstiefe 5.

Tabelle I: Vergleich der Schnittverfahren (320 × 240 Pixel, 1 Sehstrahl je Pixel, 3 Läufe)

Verfahren	**Laden [ms]**	**Min [s]**	**Median [s]**	**Mittel [s]**	**je Pixel [µs]**	**je Test [ns]**
Badouel	23	67,80	67,89	67,87	884	31,84
Möller-Trumbore	24	16,69	16,79	16,78	219	7,87

Möller-Trumbore ist um den Faktor 4,04 schneller. Die Läufe streuen um höchstens 1,1 %, der Unterschied ist also kein Messrauschen. Beide Verfahren führen dieselben 2 132 321 050 Schnitttests aus. Ein Test dauert damit 7,87 ns gegenüber 31,84 ns, bei 3,2 GHz sind das etwa 25 und 102 Takte. Der Abstand ist größer, als die Zahl der Multiplikationen erwarten lässt. Hinzu kommen bei Badouel die Wurzel und die Abhängigkeiten zwischen den Schritten. Der Schnittpunkt hängt vom Abstand ab und dieser von der Division, sodass der Prozessor die Rechnungen kaum überlappen kann. Spielraum gibt es bei beiden Verfahren. Kanten und Normale werden bei jedem Test neu berechnet und die Schnitttests liegen in einer eigenen Übersetzungseinheit, sodass sie ohne Link-Time-Optimierung nicht eingebettet werden.

Strahlen und Schnitttests. Tabelle II schlüsselt die Strahlen der Referenzkonfiguration nach ihrer Art auf.

Tabelle II: Strahlen und Schnitttests bei 320 × 240 Pixeln (Möller-Trumbore)

Strahlart	**Strahlen**	**Schnitttests**	**Tests je Strahl**
Primärstrahlen	76 800	1 384 857 600	18 032
Folgestrahlen (Spiegelung, Brechung)	18 794	338 893 408	18 032
Schattenstrahlen	24 398	408 570 042	16 746
Summe	119 992	2 132 321 050	17 771

Primär- und Folgestrahlen prüfen alle 18 032 Dreiecke. Schattenstrahlen gibt es weniger als Primärstrahlen, da manche Primärstrahlen nur den Himmel treffen und vom Licht abgewandte Flächen keinen brauchen. Der Abbruch beim ersten Treffer spart wenig. Ein Schattenstrahl prüft im Mittel 93 % der Dreiecke, da die meisten das Licht erreichen.

Skalierung mit Auflösung und Sehstrahlen je Pixel. Da jeder Strahl gleich viele Dreiecke prüft, sollte die Laufzeit proportional zur Zahl der Pixel und zur Zahl der Sehstrahlen je Pixel wachsen. Tabelle III vergleicht den gemessenen mit dem erwarteten Faktor.

Tabelle III: Laufzeit von Möller-Trumbore nach Auflösung und Sehstrahlen je Pixel (Median aus 3 Läufen)

Auflösung	**Sehstrahlen je Pixel**	**Laufzeit [s]**	**je Sehstrahl [µs]**	**Faktor gem.**	**Faktor erw.**
160 × 120	1	4,25	221,2	0,253	0,25
320 × 240	1	16,79	218,6	1,000	1,00
640 × 480	1	67,45	219,6	4,018	4,00
320 × 240	4	67,26	218,9	4,006	4,00
320 × 240	8	134,40	218,8	8,006	8,00
320 × 240	16	269,40	219,2	16,047	16,00

Die Laufzeit wächst wie erwartet linear. Keine Messung weicht um mehr als 0,4 % vom erwarteten Faktor ab. Da alle Auflösungen denselben Bildausschnitt zeigen, bleibt die mittlere Zahl der Strahlen je Pixel gleich. Zufallszahlen und das Mitteln der Farben fallen gegenüber den rund 17 771 Schnitttests je Strahl nicht ins Gewicht. Badouel wurde zusätzlich bei allen drei Auflösungen gemessen und ist dort um den Faktor 4,03 bis 4,04 langsamer, da der Unterschied im einzelnen Schnitttest entsteht.

Folgerung. Der Aufwand des Brute-Force-Raytracers wächst mit dem Produkt aus der Zahl der Strahlen und der Zahl der Dreiecke. Hochgerechnet auf die voreingestellte Auflösung von 1280 × 720 Pixeln mit 16 Sehstrahlen je Pixel ergibt sich rund das 192-Fache der Referenzmessung, also etwa 54 Minuten für ein einziges Bild. Die Wahl des Schnittverfahrens bringt nur einen festen Faktor. Eine wesentliche Beschleunigung ist nur zu erreichen, wenn ein Strahl deutlich weniger Dreiecke prüfen muss. Hinter den beiden Suchen der Welt kann dafür ein k-d-Baum die Brute-Force-Suche ersetzen.

Literatur

[1] T. Whitted, „An Improved Illumination Model for Shaded Display“, *Communications of the ACM*, Bd. 23, Nr. 6, S. 343–349, 1980.

[2] D. Badouel, „An Efficient Ray-Polygon Intersection“, *Graphics Gems*. Academic Press, S. 390–393, 1990.

[3] T. Möller und B. Trumbore, „Fast, Minimum Storage Ray-Triangle Intersection“, *Journal of Graphics Tools*, Bd. 2, Nr. 1, S. 21–28, 1997.

[4] C. Schlick, „An Inexpensive BRDF Model for Physically-based Rendering“, *Computer Graphics Forum*, Bd. 13, Nr. 3, S. 233–246, 1994.